A REHEARSAL SCRIPT, NOT A SLIDE TEMPLATE

How to present CUDAnet like you've given this talk fifty times

Twelve slides, word-for-word. What to say, why the slide exists, what the audience will ask, and what will make you look like you don't understand your own paper — so you never do that thing.

IEEE conference audience | 10–12 min target | 12 slides | assumes zero prior CUDA knowledge

Read this part first — how to use the guide

You did not just write a summary of someone else's work. You found and fixed a real, reproducible bug. Every time you feel nervous in the next ten minutes, remember that one fact: you can reproduce 0.0% accuracy on command, and you can reproduce 97.2% on command, by changing one thing. That's a stronger position than most people walk into a conference room with.

Each slide below has the same eight parts, always in this order: what's *on the slide*, what you *say* (word for word — read it a few times until it sounds like you, not like a script), *why* the slide exists in the talk's argument, the *figures* to display, any *code* you're allowed to show, *keywords* to land with emphasis, what to physically *point at*, and the *questions* people are likely to ask with your best answer ready. A final "mistakes to avoid" list closes each slide.

Timing: the full script below runs about 12 minutes 25 seconds read at a natural, unhurried pace. If your slot is a hard 10, cut Slide 3 (Related Work) down to naming two papers instead of four, and read Slide 8 (Experimental Setup) faster — those two cuts alone save about a minute and a half without touching your actual contribution.

CUDA words you need to own before you say them out loud

You don't need to be a CUDA expert to give this talk well — you need to be able to define eight words without hesitating, because the moment you hesitate on a word like "kernel" or "atomic," the audience stops trusting the rest of the talk. Read this once, out loud, the night before.

`GPU vs. CPU`

SIMPLY:

A CPU is a handful of very smart workers who can each do complicated things one after another, fast. A GPU is thousands of much simpler workers who can each only do one simple thing — but they all do it at the exact same instant.

TECHNICALLY:

A CPU has a few (4–64) powerful cores optimized for low-latency sequential instruction streams; a GPU has thousands of simpler cores (3,328 on the RTX A2000 used in this paper) optimized for high-throughput data-parallel execution of the same instruction across many data elements — exactly the shape of "run this same forward pass on every image in the batch."

`Thread`

SIMPLY:

One single worker. In this paper, one thread usually handles one training image.

TECHNICALLY:

The smallest unit of execution on a GPU — a single instruction stream with its own registers and program counter, one of thousands launched together to run identical code over different data.

`Kernel`

SIMPLY:

The one small program that every thread runs. You write it once; the GPU runs thousands of copies of it at the same time.

TECHNICALLY:

A function marked `__global__` in CUDA C, launched from the CPU with a specified number of threads, executed identically (but on different data) by every one of them.

`Block & grid`

SIMPLY:

Threads are organized into small teams called blocks, and all the blocks together are called a grid — like a stadium organized into sections, and the sections together make the whole crowd.

TECHNICALLY:

A *block* is a group of threads (this paper uses 128 or 256 per block) that can synchronize with each other and share fast on-chip memory; a *grid* is the full set of blocks launched by one kernel call. Blocks cannot synchronize with each other mid-kernel — this fact is exactly why the race condition in this paper is possible.

`Race condition`

SIMPLY:

Two workers try to update the same shared notebook at the same time. Both read "100," both add their own number, both write back — whoever writes last erases the other's contribution, and nobody notices.

TECHNICALLY:

Undefined behavior arising when two or more threads access the same memory location concurrently, at least one write, with no enforced ordering — the final value depends on non-deterministic scheduling rather than program logic.

`Atomic operation (atomicAdd)`

SIMPLY:

A single action that's guaranteed to happen all at once, with no other worker able to interrupt it halfway through — like a bank teller who won't start your neighbor's transaction until yours is fully done.

TECHNICALLY:

A hardware-guaranteed indivisible read-modify-write instruction. Critically, it only makes *that one operation* indivisible — a multi-step formula built from several reads and writes around it is *not* automatically made safe just because one step inside it uses an atomic.

`Synchronization barrier (cudaDeviceSynchronize())`

SIMPLY:

A checkpoint where you make everyone wait until every worker has finished the current task, before anyone starts the next one.

TECHNICALLY:

A host-side (CPU-side) call that blocks until every kernel previously launched on the device has completed execution — the only reliable way to guarantee that every thread's writes are visible before a dependent kernel reads them.

`Throughput / speedup`

SIMPLY:

How much work gets done per second, and how many times faster that is than the slow way of doing it.

TECHNICALLY:

Speedup = $T_{baseline}$ / $T_{accelerated}$, measured here in wall-clock seconds per training epoch via CUDA event timers, compared against a serial CPU implementation of the identical algorithm.

WHAT'S ON THE SLIDE

Paper title, your name and co-authors, affiliation (Manipal Institute of Technology Bengaluru), and one short subtitle line under the title — something like *"A race condition in GPU-parallel optimizers, found and fixed."* Nothing else. No bullet points. Let it breathe.

SAY THIS, WORD FOR WORD

```
Good [morning/afternoon]. My name is [your name], and together with my co-authors Chiranth R and Dr. Pavithra N from Manipal Institute of Technology, I'm presenting CUDAnet — a neural network training engine we built entirely from scratch in CUDA C, with zero dependency on frameworks like PyTorch or TensorFlow. In the next few minutes, I'll show you a subtle bug we found in how GPUs parallelize modern optimizers — one that silently destroys training — and the fix we designed for it.
```

WHY THIS SLIDE EXISTS

A conference audience decides in the first fifteen seconds whether to actually listen. Naming the punchline early — "a bug that silently destroys training" — is a hook. Don't save your best line for the end; nobody remembers slide 12 if they checked out during slide 2.

FIGURES TO SHOW

none

CODE TO SHOW

None.

KEYWORDS TO LAND

`from-scratch CUDA` `zero dependency`

WHAT TO POINT AT

👉 Nothing — look at the audience, not the screen, for this slide. It's the one moment you don't need the slide's help.

LIKELY QUESTIONS HERE

Almost never questioned at the title slide — move on confidently.

MISTAKES TO AVOID

- ✕ Don't read the title off the slide in a flat monotone — the audience can already read.
- ✕ Don't apologize or hedge ("um, so, I guess I'll start…"). Start with your first sentence, already decided in advance.
- ✕ Don't rush this slide just because "it's just the title" — it sets your vocal pace for the whole talk.

SLIDE 02 / 12

The problem

≈ 1:05

WHAT'S ON THE SLIDE

One headline sentence, large: "**Parallelizing training is easy. Parallelizing the optimizer is not.**" Below it, a simple two-part visual you sketch yourself (no figure in the paper covers this) — on the left, many small boxes labeled "Thread 1, Thread 2, … Thread 2048," each with an arrow pointing at one shared box on the right labeled "v (velocity)". That's the whole slide.

SAY THIS, WORD FOR WORD

```
Let's start with why this problem exists at all. When you train a neural
network on a GPU, you split your mini-batch across thousands of GPU
threads — think of a thread as one tiny independent worker, and a kernel
as the small program every worker runs at the same time. Each thread takes
one training image, runs it forward through the network, and computes how
wrong the prediction was. That part is what we call "embarrassingly
parallel" — every thread's work is completely independent, so running two
thousand of them at once is completely safe. But then comes the optimizer
step — the part that actually updates the network's weights using that
error signal. And optimizers like Momentum and Adam are not stateless.
They keep a running memory — velocity for Momentum, moment estimates for
Adam — that carries over from one training step to the next. All two
thousand threads want to update that same shared memory at the same time.
And that's where things quietly go wrong.
```

WHY THIS SLIDE EXISTS

Every good conference talk earns its solution by first making the audience feel the problem. This slide plants the one idea the rest of the talk depends on: some parts of training are safe to parallelize, some aren't, and the difference is whether there's shared, persistent state.

FIGURES TO SHOW

`none from the paper — draw your own two-box diagram`

CODE TO SHOW

None yet — save code for Slide 5, where the bug becomes concrete.

KEYWORDS TO LAND

`embarrassingly parallel` `stateful` `shared memory`

WHAT TO POINT AT

👉 The cluster of "Thread" boxes when you say "two thousand threads."

👉 The single shared "v" box when you say "that same shared memory."

LIKELY QUESTIONS

Q — What is a race condition, in one sentence?

A — It's when two or more threads read and write the same memory at the same time without coordination, so the result depends on unpredictable timing rather than the intended computation — and one thread's contribution can be silently erased by another's.

Q — Isn't this just a threading problem that exists on CPUs too?

A — The underlying hazard is the same concurrency problem you'd see on a multi-core CPU — but the scale is different by orders of magnitude. A CPU might have 16 threads racing; here we have 2,048, and the GPU offers no cross-block synchronization primitive mid-kernel, so the exposure is both bigger and harder to guard against.

MISTAKES TO AVOID

✗ Don't say the phrase "race condition" before you've explained what it means — you'll lose anyone who isn't already a systems person.

✗ Don't dive into CUDA syntax here. This slide is conceptual; save every piece of code for Slide 5.

✗ Don't rush — this is the idea the entire talk is built on. If the audience doesn't get this slide, nothing after it lands.

SLIDE 03 / 12

Related work

≈ 0:50

WHAT'S ON THE SLIDE

A short list, four lines max: *Wang et al. — 17× inference speedup. Rao et al. — 487× backprop speedup. Zhao et al. — 110× speedup, 97.2% MNIST accuracy.* Under the list, one bolded closing line: "**None of these identify or resolve race conditions in stateful optimizers.**"

SAY THIS, WORD FOR WORD

This isn't the first paper to accelerate neural network training with hand-written CUDA kernels. Wang et al. reported a 17× inference speedup, Rao et al. 487× for backpropagation, Zhao et al. 110× with 97.2% MNIST accuracy — genuinely impressive engineering. But every one of these only parallelizes the stateless parts: the forward pass, the backward pass, plain gradient descent. None of them puts a stateful optimizer like Adam inside a per-sample GPU kernel, so this particular race condition never had a chance to surface in their designs. Frameworks like PyTorch avoid the problem by construction — the optimizer step is always a separate function call from the backward pass — which means nobody using a framework ever sees why the naive version would be wrong in the first place.

WHY THIS SLIDE EXISTS

A conference reviewer's first instinct is "hasn't someone already done this?" This slide answers that question before anyone has to ask it, and positions your contribution precisely: not faster kernels, but a correctness argument nobody else made explicit.

FIGURES TO SHOW

none — this is a citations slide

CODE TO SHOW

None.

KEYWORDS TO LAND

`stateless parts` `frameworks hide the mechanism`

WHAT TO POINT AT

👉 Each citation as you name it — don't just read the list, gesture along it left to right.

LIKELY QUESTIONS

Q — How is this different from just calling `optimizer.step()` separately in PyTorch?

A — Conceptually it's the same shape — separate the gradient phase from the state-update phase. Our contribution is making that separation explicit, naming the failure mode it prevents, and demonstrating with real accuracy numbers exactly what breaks if you skip it. Framework users get this for free and never see why it matters; we make the mechanism and its necessity visible.

Q — What about asynchronous / distributed training work, like Dean et al.'s DistBelief?

A — Those systems sidestep the correctness question rather than answering it — they tolerate "stale" gradients, where a worker updates against slightly out-of-date weights. Our approach is fully synchronous and zero-staleness: every update uses the complete, current batch gradient, with no approximation.

MISTAKES TO AVOID

- ✕ Don't spend more than a minute here — related work is a required beat, not the show.
- ✕ Don't say "no one has ever done this" — say precisely what's missing (explicit identification of this race condition), which is defensible; a blanket novelty claim isn't.

SLIDE 04 / 12

System architecture

≈ 1:00

WHAT'S ON THE SLIDE

Figure 1 from the paper, full width. Nothing else — let the diagram do the work.

SAY THIS, WORD FOR WORD

Here's the system we built — CUDAnet. It's a fully configurable multi-layer perceptron, shown here in its default MNIST configuration: 784 input pixels from a 28-by-28 image, two hidden layers of 128 neurons with ReLU and batch normalization, and a 10-way softmax output. Every weight and bias lives in one flat array on the GPU, addressed by precomputed offsets rather than pointers — good for memory efficiency, though I won't dwell on that today. What matters for this talk is the split you see here: Phase 1, where thousands of threads run forward and backward propagation in parallel and accumulate gradients; and Phase 2, a single, sequential, race-free step that actually updates the optimizer state. That two-phase split is the heart of this paper, and the next slides explain exactly why it's necessary.

WHY THIS SLIDE EXISTS

You need one concrete picture of the system before the audience can follow an abstract correctness argument. This slide also plants "Phase 1 / Phase 2" visually, so when you name Two-Phase Gradient Accumulation on Slide 6, the audience already has a picture to hang it on.

FIGURES TO SHOW

Fig. 1 — required

CODE TO SHOW

None — this is the one purely visual slide in the deck.

KEYWORDS TO LAND

`multi-layer perceptron` `batch normalization` `Phase 1 / Phase 2`

WHAT TO POINT AT

👉 Trace input → hidden 1 → hidden 2 → output left to right as you describe the layers.

👉 Physically point at the Phase 1 box, pause, then point at the Phase 2 box.

LIKELY QUESTIONS

Q — Why an MLP and not a CNN, given MNIST is usually solved with convolutions?

A — The architecture choice is deliberate: an MLP isolates the parallel-correctness question we care about from the added complexity of convolutional kernels. We're not claiming state-of-the-art accuracy — we're claiming a systems result, and the simplest architecture that still has a real stateful optimizer is the right instrument for that.

Q — Is the network size fixed, or configurable?

A — Fully configurable at runtime — hidden layer count and width are command-line arguments, not compile-time constants. 784-128-128-10 is simply the configuration we benchmarked and report in the paper.

MISTAKES TO AVOID

✗ Don't get pulled into explaining the flat-array memory layout in detail — it's real engineering but it's not this talk's contribution. One sentence, then move on.

✗ Don't call this "just a normal neural network" dismissively — it's the concrete instrument the whole result rests on.

SLIDE 05 / 12

The race condition

≈ 1:25

WHAT'S ON THE SLIDE

The naive Momentum update equation (paper's Eq. 4), rendered large: `v_t = βv_t + ∇W L`. Below it, the tiny code snippet at right. In the corner, a callout box: "**Momentum accuracy: 97.2% → 0.0%.**"

SAY THIS, WORD FOR WORD

Now here's the actual bug. Imagine we write the Momentum update exactly the way the math tells us to, but inside a kernel where every one of 2,048 threads — one per training sample — runs this same line at once. Thread i reads the shared velocity value, computes its own updated version using its own gradient, and writes it back. Thread j does the exact same thing, at almost the same instant, using the same old velocity value it read a moment ago. Whichever thread writes last simply overwrites the other — not delays it, erases it. There's no error message, no crash, no compiler warning. The code runs, the network trains, the loss even looks like it's moving at first. But the optimizer's memory has been silently corrupted on every one of roughly eighteen thousand weights, every single batch. And the empirical result is not subtle: Momentum's accuracy collapses from 97.2 percent to 0.0 percent — indistinguishable from a network that was never trained at all.

WHY THIS SLIDE EXISTS

This is the emotional and intellectual center of the talk — the "wait, that's a real bug" moment. Everything before this slide was setup; everything after it is resolution. Give it the time it needs and let the "0.0%" land with a pause before you move on.

FIGURES TO SHOW

`Eq. 4 from the paper` `optional: Table II "before" column only`

CODE TO SHOW

```
// naive (WRONG) — one thread per SAMPLE, all racing on the same v
v = beta * v + grad_i;   // read, then compute, then write — three separate steps
theta -= lr * v;          // another thread may have already changed v by now
```

KEYWORDS TO LAND

`read-modify-write` **`silently corrupted`** **`0.0% accuracy`**

WHAT TO POINT AT

👉 The v symbol appearing in both "Thread i" and "Thread j" — same variable, two readers.

👉 The "0.0%" number — point, then pause for a full second before continuing.

LIKELY QUESTIONS

Q — Why doesn't `atomicAdd` already fix this?

A — `atomicAdd` only makes the addition itself indivisible — it guarantees no other thread interrupts that one add. But the Momentum formula is "read the old velocity, multiply by beta, add the gradient, write back" — several steps. An atomic protects one step, not the whole sequence, so the read and the write can still be split apart by another thread's operation in between.

Q — Does this affect Adam too, or just Momentum?

A — Any optimizer with persistent, cross-step state is affected — Momentum's velocity, Adam's first and second moment estimates, RMSProp, LAMB. Plain SGD has no such state, so it's the one exception, which is actually part of why the bug is easy to miss: if you test SGD first, everything looks correct.

Q — How did you first notice something was wrong?

A — The signature was unmistakable: Momentum and Adam collapsed to exactly the accuracy of random guessing on a 10-class problem — about 10 percent, or in some runs even lower — while the loss curve and code both looked superficially normal. That specific failure mode pointed straight at optimizer-state corruption rather than a hyperparameter or data issue.

MISTAKES TO AVOID

- ✕ Don't show messy real source code here — three lines of clean pseudocode teaches the bug faster than forty lines of your actual kernel.
- ✕ Don't call it "just a bug" — frame it as a subtle, load-bearing correctness issue that compiles clean and produces no error, which is precisely what makes it dangerous.
- ✕ Don't explain CUDA's hardware-level atomic implementation in depth here — that's a rabbit hole this audience doesn't need; keep the focus on the read/write ordering problem.

SLIDE 06 / 12

The fix: Two-Phase Gradient Accumulation

≈ 1:35

WHAT'S ON THE SLIDE

A simplified version of Algorithm 1 — two labeled boxes, "Phase 1" and "Phase 2," with the code snippet underneath each. Name the technique in large text at the top: "**Two-Phase Gradient Accumulation (2PGA).**"

SAY THIS, WORD FOR WORD

Our fix doesn't add locks, doesn't add warp-level tricks, and doesn't tolerate stale data. It changes who owns what memory, at two different points in the batch. In Phase 1, every thread still runs in parallel, but the only thing it's allowed to do to shared memory is add a number to it, using CUDA's atomicAdd — which is genuinely safe here, because addition is commutative: it doesn't matter what order two thousand additions happen in, the sum comes out the same either way. Once every thread has finished — enforced by a single hardware synchronization barrier — we launch a second kernel. But this time, instead of one thread per training sample, we launch one thread per weight. Thread k touches weight k, and nothing else. Nobody else is racing it. So the full Momentum or Adam formula — the one with the read, the multiply, the write — runs exactly once, by exactly one owner, with zero concurrent writers. No atomics are even needed in this phase, because there's no race left to arbitrate. And I want to stress this: this is not slower mini-batch SGD in disguise. Phase 1 already produced the correct summed gradient before our fix and after it — nothing changes there. The entire fix lives in Phase 2 alone.

WHY THIS SLIDE EXISTS

This is the paper's central contribution — give it the most rhetorical weight in the whole talk. The goal is for one phrase to survive in the audience's memory after the talk ends: "Two-Phase Gradient Accumulation — many writers for the sum, one owner for the state."

FIGURES TO SHOW

`Fig. 1 again (callback)` `Algorithm 1`

CODE TO SHOW

```
// Phase 1 — many threads, safe because "+" commutes
atomicAdd(&grad_w[i], sample_gradient);

// barrier — wait for every thread to finish
cudaDeviceSynchronize();

// Phase 2 — ONE thread owns EACH weight; no race possible
g = grad_w[i] / batch_size;
v = beta * v + g;
theta[i] -= lr * v;
```

KEYWORDS TO LAND

`commutative` `single owner` `synchronization barrier` `Two-Phase Gradient Accumulation`

WHAT TO POINT AT

👉 The `atomicAdd` line, then trace down to the barrier line.

👉 "one thread per weight" — hold your point there for a beat, it's the key sentence.

LIKELY QUESTIONS

Q — Doesn't the synchronization barrier cost you performance?

A — Measurably, but small. For our 128×128 network, Phase 2 touches about 18,000 weights and finishes in under 0.01 milliseconds — under half a percent of one epoch's time. And the argument scales: however many parameters you have, you assign one thread per weight across enough blocks; "one owner per address" stays true no matter the parameter count, which is why we frame this as compatible with much larger, Transformer-scale models.

Q — Does this generalize beyond Momentum and Adam?

A — Yes — the decomposition doesn't depend on the specific update formula. It depends only on separating what's commutative (gradient summation) from what isn't (a stateful recurrence). Any optimizer with cross-step state — RMSProp, LAMB, Adagrad — has the identical failure mode and the identical fix.

Q — Why not just use a mutex or lock instead?

A — Locks would serialize access to shared state, but at massive throughput cost — thousands of threads contending for the same lock on the same address is a severe bottleneck on a GPU. 2PGA gets the same correctness with no locking at all, because Phase 2 removes the contention structurally rather than arbitrating it.

MISTAKES TO AVOID

- ✕ Don't undersell this as "we just moved some code around" — the insight is realizing *which operations commute and which don't*, and restructuring thread ownership around that distinction.
- ✕ Don't forget to actually say the name "Two-Phase Gradient Accumulation" out loud, clearly, at least once — it's the one term you want people repeating back to you afterward.
- ✕ Don't let this slide run past 100 seconds even though it's the most important one — a long, meandering explanation of your best idea is worse than a tight one.

SLIDE 07 / 12

Correct batch normalization on GPU

≈ 0:55

WHAT'S ON THE SLIDE

The BN equations (paper Eq. 2–3) at the top, small. Below, a simple five-box pipeline diagram you draw yourself: `linear → sync → stats → normalize → activate`, each box connected by an arrow, with the sync step visually distinct (e.g. a vertical bar or gate icon) from the rest.

SAY THIS, WORD FOR WORD

Batch normalization adds one more correctness wrinkle. To normalize a neuron's values, you need the mean and variance of that neuron across the entire batch — which means every one of 2,048 samples' values has to already be written to memory before anyone can safely read them to compute a mean. A single, one-shot kernel launch can't guarantee that, because different thread blocks run in an undefined order relative to each other, with no built-in way to signal "everyone's done." So we broke the batch-norm forward pass into a short pipeline of small kernels — compute the linear layer, synchronize, compute the batch statistics with a parallel reduction, apply the normalization, then activate — and mirrored the same structure going backward. Each synchronization point in between guarantees every sample's contribution is visible before the next kernel reads it.

WHY THIS SLIDE EXISTS

It's the paper's second contribution, and it reuses the exact same "you need a barrier before anyone reads shared results" logic from Slide 6 — which makes it feel like reinforcement rather than a brand-new idea, and keeps this slide brisk.

FIGURES TO SHOW

Eq. 2–3 · your own pipeline sketch (no paper figure for this)

CODE TO SHOW

None required — if you have extra time, one line naming the kernel sequence is enough: `fwd_linear → bn_stats → bn_apply → activation`.

KEYWORDS TO LAND

`batch statistics` `parallel reduction` `explicit barrier`

WHAT TO POINT AT

👉 Trace along your pipeline diagram left to right, pausing visibly at the sync gate.

LIKELY QUESTIONS

Q — Why not just synchronize once at the very end instead of once per layer?

A — Because each hidden layer's batch statistics depend on that layer's own linear output, which depends on the previous layer already being fully normalized and activated. The dependency chain is layer by layer, so the barrier has to be too — one synchronization at the end would let later layers read incomplete earlier results.

Q — Does this add much overhead compared to training without batch norm?

A — Yes, measurably — our BN configurations run about 25–30% slower per epoch than the equivalent non-BN configuration, from the extra kernel launches and synchronization points. We see that as future kernel-fusion work rather than a flaw in the correctness argument itself.

MISTAKES TO AVOID

- ✕ Don't let this slide compete with Slide 6 for the audience's attention — keep it short and clearly secondary in the talk's structure.
- ✕ Don't derive the batch-norm backward-pass math on the slide — name that it exists (the closed-form Ioffe–Szegedy identity) without deriving it live.

SLIDE 08 / 12

Experimental setup

≈ 0:35

WHAT'S ON THE SLIDE

A compact two-column key/value list — no full sentences: GPU: RTX A2000, 12GB, sm_86 · Compiler: nvcc -O2 + g++-12 · Dataset: MNIST, 60k/10k · Network: 2×128 · Epochs/Batch: 20 / 2048 · Split: 48k train / 12k val / 10k test · Init: Xavier, seed 42.

SAY THIS, WORD FOR WORD

> Quickly, on methodology: everything ran on a single NVIDIA RTX A2000 with 12 gigabytes of memory, Ampere architecture, compiled with nvcc and g++-12. We use standard MNIST — 60,000 training and 10,000 test images — with an 80/20 split carving out 12,000 validation images, so every epoch is a genuine full pass over 48,000 training samples in 23 mini-batches of 2,048. All experiments use the same two-hidden-layer, 128-neuron network, trained for 20 epochs, with Xavier initialization and a fixed random seed for reproducibility.

WHY THIS SLIDE EXISTS

A required conference-talk beat — reviewers need to know your results are reproducible and fairly measured — but it's not where your value is. Read it cleanly and move on; don't let methodology eat into your results time.

FIGURES TO SHOW

`none — key/value table only`

CODE TO SHOW

None.

KEYWORDS TO LAND

`RTX A2000` `23 mini-batches / epoch`

WHAT TO POINT AT

👉 Nothing special — this slide should feel like a fast, confident recitation, not a pointing exercise.

LIKELY QUESTIONS

Q — Why 2,048 as the batch size specifically?

A — We profiled batch-size scaling separately (reported in the paper's Table V): GPU utilization keeps climbing through 4,096, but 2,048 is the practical sweet spot on this GPU — past it, Phase 2's synchronization overhead starts growing faster than the throughput gain, about 17% more epoch time at 4,096 versus 2,048.

Q — Only one GPU, one seed — how confident are you in these numbers?

A — Fair challenge — this is a single-GPU, single-seed study, and we say so directly in the paper's limitations. What we're confident about is the qualitative result: the exact same code, with only the parallelization strategy changed, moves from 0% to 97%. That's not a borderline effect sensitive to a particular seed.

MISTAKES TO AVOID

❌ Don't spend more than 40 seconds here even if you feel like justifying every hyperparameter — save that depth for Q&A, where it'll actually be asked for.

SLIDE 09 / 12

Results I — recovery from the race condition

≈ 1:10

WHAT'S ON THE SLIDE

Figure 4 (grouped before/after bar chart), full width, with the four recovery deltas (+97.2 pp, +97.2 pp, +63.1 pp) visible as data labels on the bars.

SAY THIS, WORD FOR WORD

Here's the headline result. Before our fix, Momentum and Adam both collapse to zero percent accuracy — literally random guessing on a ten-class problem. After the fix, both recover to 97.2 percent — a 97.2 percentage-point swing, from a completely broken network to our best-performing configuration, with no other change to the code. Batch-norm-plus-Adam shows the same pattern at a different scale: 17.9 percent before, 81 percent after — a real recovery, even though that configuration has its own separate story, which I'll come back to. SGD, which has no persistent optimizer state, is completely unaffected by this fix — its own improvement to 94.3 percent comes from an unrelated change, moving to full-dataset epochs. That's actually a useful control: it confirms the fix is doing exactly what we claim, and nothing else.

WHY THIS SLIDE EXISTS

This is the payoff — deliver the paper's strongest, most quotable numbers with total confidence. Let "zero to ninety-seven point two" land as a full sentence on its own, with a pause on either side.

FIGURES TO SHOW

Fig. 4 — required

reference Table II verbally

CODE TO SHOW

None — this is a pure results slide.

KEYWORDS TO LAND

`0.0% → 97.2%` `unaffected (control)`

WHAT TO POINT AT

👉 Each before/after bar pair as you name its numbers — hatched bar first, solid bar second.

LIKELY QUESTIONS

Q — Could the 0% before-fix number be caused by something else, like a bad learning rate?

A — No — we used the exact same learning rate, initialization, and seed in both runs; the only variable that changed is whether the optimizer update is applied per-sample (racy) or in the two-phase form. And 0% is a very specific signature — it isn't "worse convergence," it's exactly what you'd expect from an optimizer state that's pure noise.

Q — Is 0% reproducible, or a one-time fluke?

A — Reproducible — it's deterministic given the race, not a rare event; with 2,048 threads racing on every one of roughly 18,000 weights every batch, the corruption isn't an edge case, it's the expected outcome of that design.

MISTAKES TO AVOID

- ✕ Don't rush past the "0.0% to 97.2%" number — it's the single most important sentence in your talk; say it deliberately, not as a list item.
- ✕ Don't gloss over the BN+Adam+LR number here without flagging you'll return to it — an audience member doing the math (81% is well below 97.2%) will otherwise wonder if you noticed.

SLIDE 10 / 12

Results II — speed & optimizer comparison

≈ 1:25

WHAT'S ON THE SLIDE

Figure 3 (log-scale time/speedup chart) on the left half; a condensed 4-row version of Table III (SGD / Momentum / Adam / Adam+LR final accuracy only — drop the BN rows here, they belong to Slide 11) on the right half.

SAY THIS, WORD FOR WORD

On raw throughput: our serial CPU baseline takes about 2,070 seconds per epoch; every CUDA configuration finishes in roughly one second — SGD at 0.94 seconds is a 2,202-times speedup, and even our heaviest configuration still holds well over a thousand-times speedup. On accuracy, Momentum and Adam are statistically tied at 97.2 percent, both roughly 3 points above plain SGD, and both converge to within about a point of their final accuracy within just 5 of the 20 epochs — Adam's adaptive per-weight step size gives it a real head start early on.

WHY THIS SLIDE EXISTS

Standard "how fast, how accurate" conference beat. Pick three or four headline numbers and say them with confidence rather than trying to narrate every cell of every table — a wall of numbers read aloud is the fastest way to lose an audience's attention.

FIGURES TO SHOW

Fig. 3 — required

Table I / III (condensed)

CODE TO SHOW

None.

KEYWORDS TO LAND

`2,202× speedup` `97.2%` `adaptive moment estimates`

WHAT TO POINT AT

👉 The log-scale axis on Fig. 3 — explicitly note it's log scale, and briefly say why (the gap between CPU and GPU times spans three orders of magnitude; a linear axis would make every GPU bar look like zero).

LIKELY QUESTIONS

Q — Is the CPU baseline a fair comparison?

A — We measured a completed 20-epoch serial run to verify this — it averages about 2,070 seconds per epoch, matching our original projection almost exactly, so the reported speedups hold up against a real completed measurement, not just an estimate.

Q — Why does Adam need a smaller learning rate (0.01) than SGD or Momentum (0.1)?

A — Adam's per-weight adaptive scaling — dividing by the square root of a running second-moment estimate — already amplifies small, consistent gradients, so a learning rate tuned for plain SGD would overshoot badly with Adam; 0.01 is the standard order-of-magnitude choice in the literature and what we found stable in practice.

MISTAKES TO AVOID

✗ Don't read every row of every table — pick your best three or four numbers and deliver them with weight; the slide itself carries the rest for anyone who wants detail.

✗ Don't skip explaining the log scale — an unexplained log axis reads as either a mistake or an attempt to exaggerate the speedup, and you want neither impression.

The BN+Adam anomaly — an honest limitation

≈ 0:55

WHAT'S ON THE SLIDE

Figure 5 (BN impact bar chart), plus one annotated line under it: "**Peaks 88.1% at epoch 2 → declines to 81.0% by epoch 20.**"

SAY THIS, WORD FOR WORD

I want to spend one slide being upfront about a result that didn't go the way we expected. When we combine batch normalization with Adam and a learning-rate schedule, accuracy actually peaks early — 88.1 percent at epoch 2 — and then steadily declines to 81 percent by epoch 20. That's not slow convergence, that's instability: something is actively getting worse over training, not just improving slowly. We checked whether the learning-rate schedule itself was the cause by switching from a per-epoch to a per-batch cosine decay, and that only recovered about one point — so the schedule isn't the main culprit. Our leading hypothesis is a genuine interaction between Adam's evolving moment estimates and batch normalization's own evolving statistics — two different kinds of memory that may be working against each other during training. We think a batch-norm-aware optimizer, with decoupled learning rates for the normalization parameters, is the right next step, and we've left it as future work rather than smoothing it over.

WHY THIS SLIDE EXISTS

Conference audiences trust a presenter more, not less, for reporting an inconvenient result clearly. This slide also pre-empts the single most likely tough question — "why is your flagship configuration your worst result?" — by answering it before anyone has to ask.

FIGURES TO SHOW

`Fig. 5 — required` `Table IV (epoch progression), optional`

CODE TO SHOW

None.

KEYWORDS TO LAND

`instability, not slow convergence` `BN+Adam interaction`

WHAT TO POINT AT

👉 The peak of the curve at epoch 2, then trace the decline with your finger all the way to epoch 20 — the shape itself is the argument.

LIKELY QUESTIONS

Q — If BN+Adam+LR is your worst configuration, why include it as a flagship feature at all?

A — Because the paper's contribution isn't "BN+Adam+LR is the best configuration" — it's the two-phase gradient accumulation mechanism, which BN+Adam+LR also depends on and benefits from equally. We report this result specifically because omitting an inconvenient number would undermine the credibility of every other accuracy figure in the paper.

Q — Have you tried simply lowering Adam's learning rate for this configuration?

A — Not exhaustively — that's exactly the kind of decoupled-learning-rate direction we flag as future work. Our diagnosis so far rules out the LR *schedule* as the main cause (the per-batch cosine fix only recovered about a point); a smaller base LR specifically for BN parameters is a distinct, untested hypothesis.

MISTAKES TO AVOID

❌ Don't get defensive or apologetic — deliver this as evidence of rigor, in the same confident tone as your best number.

❌ Don't try to explain it away with a hand-wavy guess stated as fact — say clearly that it's a hypothesis, not a proven diagnosis.

SLIDE 12 / 12

Edge AI case, conclusion & future work

≈ 1:10

WHAT'S ON THE SLIDE

A condensed 3-row version of Table VI (Test Accuracy / Peak VRAM / Install Size, CUDAnet vs. PyTorch) at the top. Below it, three future-work bullets: Tensor Cores (WMMA), convolutional layers (im2col), multi-GPU (NCCL). A final line: "**Thank you.**"

SAY THIS, WORD FOR WORD

One more result before I close. Because CUDAnet has no framework dependency at all — just libcuda — the compiled binary is under 2 megabytes, versus roughly 1.8 gigabytes for an equivalent PyTorch install, and its peak VRAM footprint is about 470 kilobytes versus 248 megabytes. PyTorch is faster per epoch and slightly more accurate, thanks to cuDNN and cuBLAS under the hood — we're not claiming otherwise — but on a memory-constrained edge device, that five-hundred-times difference in footprint is the entire ballgame. To conclude: we identified a genuine, silent correctness bug in how stateful optimizers get parallelized on GPUs, we fixed it with a general two-phase accumulation pattern that costs less than half a percent of runtime, and we validated it end to end on real hardware with a two-thousand-times speedup over a serial baseline. For future work, we're looking at wiring up true Tensor Core kernels for a further two to three times speedup, extending to convolutional layers via im2col to push past MNIST toward CIFAR-10, and scaling across multiple GPUs with NCCL — which, importantly, requires no change at all to the core two-phase correctness argument. Thank you — I'm happy to take questions.

WHY THIS SLIDE EXISTS

Closes with the practical "why this matters beyond a benchmark" (edge deployment), then a tight restatement of the contribution, ending on an explicit, natural handoff into Q&A rather than trailing off.

FIGURES TO SHOW

`Table VI (condensed)`

CODE TO SHOW

None.

KEYWORDS TO LAND

`under 2 MB` `libcuda.so` `NCCL AllReduce`

WHAT TO POINT AT

👉 The VRAM and install-size numbers side by side — let the visual gap do the persuading.

LIKELY QUESTIONS

Q — Have you actually deployed this on a Jetson or other edge device?

A — Not yet in this paper — the footprint comparison is measured on our development GPU, and Jetson deployment is exactly the next validation step we describe under future work. We'd want to confirm the same binary-size and VRAM advantages hold on that hardware before making a stronger deployment claim.

Q — What's the single most important takeaway you want us to leave with?

A — That parallel correctness is not the same as sequential correctness — an algorithm that's mathematically right on paper can still corrupt itself the moment you parallelize it naively, and Two-Phase Gradient Accumulation is a general, low-cost pattern for any stateful optimizer that needs to run safely across thousands of GPU threads.

MISTAKES TO AVOID

- ✕ Don't introduce brand-new numbers in your very last sentence — give the audience a beat to sit with the closing thought before you say "thank you."
- ✕ Don't just trail off ("...so yeah, that's basically it") — end on the explicit, rehearsed line above and make eye contact as you say it.

BONUS

If someone in the audience goes off-script

These aren't in the paper's own text, but they're things I found while reading your actual source code during an earlier deep review. A sharp question could land on any of them — you don't need to volunteer these, but you should never look surprised if asked.

"Does your Tensor Core (WMMA) build actually use Tensor Cores?"
Honest answer: not currently. The true WMMA kernels exist in the source and are architecturally correct, but a preprocessor flag issue means the `cuda_wmma` binary doesn't actually engage them yet — this is exactly why the paper correctly lists Tensor Core integration as future work rather than a delivered result. If asked directly: "That's accurate — the WMMA kernels are implemented but not yet wired into the default build; validating that path is one of our concrete next steps."

"Does your Adam implementation's bias-correction step counter increment per mini-batch, as the standard algorithm assumes?"
Honest answer: it currently increments once per epoch rather than once per mini-batch, which is a minor deviation from canonical Adam's bookkeeping. If asked: "Good catch — our current implementation ties the bias-correction step to the epoch counter rather than the mini-batch counter; it's a bookkeeping detail we're aware of and plan to align with the canonical formulation."

"**Is everything in your public repository consistent with these numbers?**"
Yes, as of your latest update — the top-level README was revised to match the actual `results/` logs and the paper exactly, including correctly identifying Momentum and Adam (not BN+Adam+LR) as the best-performing configurations.

Rehearsal script for the CUDAnet IEEE presentation. Built directly from the manuscript's six sections, all five figures, and all six tables — every number quoted above traces back to the paper or the project's own result logs.